

OBSERVATION TASK 1: JPP FUNDAMENTALS AND OOP

PROGRAM 1: JAVA PROGRAMMING FUNDAMENTALS

Title

Student Utility Program

Duration

2 Hours

Objective

To understand and implement the basic programming constructs of Java, including:

- Java program structure
- Variables and data types
- Operators
- Input and output
- Conditional statements
- Looping statements
- User-defined methods

Concepts Covered

1. Java Program Structure
2. Variables and Data Types
3. Operators
4. Input and Output
5. `if`, `if-else`, and `switch`
6. `for`, `while`, and `do-while`
7. Parameterized methods
8. Return values
9. `Scanner` for user input

Task Description

Develop a menu-driven **Student Utility Program** that performs the following operations.

Part A – Java Basics

- Display student information.
- Accept name, roll number, and marks.
- Calculate total marks.
- Calculate percentage.
- Display the result.

Part B – Conditional Statements

- Check whether a number is even or odd.
- Find the largest of three numbers.
- Display grade based on percentage.
- Display day of the week using `switch`.

Part C – Looping Statements

- Print multiplication tables.
- Display numbers from 1 to N.
- Calculate the sum of first N natural numbers.
- Generate Fibonacci series.

Part D – Methods

Create parameterized methods that return results for:

- Factorial
- Prime number checking
- Maximum of two numbers
- Area of a circle

Algorithm / Logic

1. Import `java.util.Scanner`.
2. Create the `StudentUtilityProgram` class.
3. Define methods for factorial, prime checking, maximum, and circle area.
4. Create a `Scanner` object for input.
5. Display a menu using `do-while`.
6. Read the user's choice.
7. Use `switch` to execute the selected operation.
8. For conditional operations, use `if-else`.
9. For repetitive operations, use `for` loops.

10. Call the appropriate user-defined method for method-based operations.
11. Continue displaying the menu until the user selects Exit.
12. Close the `Scanner`.

Source Code

```
package recordprograms;

import java.util.Scanner;

public class StudentUtilityProgram {

    // Method to calculate factorial
    public static int factorial(int n) {
        int fact = 1;

        for (int i = 1; i <= n; i++) {
            fact *= i;
        }

        return fact;
    }

    // Method to check prime number
    public static boolean isPrime(int n) {
        if (n <= 1)
            return false;

        for (int i = 2; i <= Math.sqrt(n); i++) {
            if (n % i == 0)
                return false;
        }

        return true;
    }

    // Method to find maximum of two numbers
    public static int maximum(int a, int b) {
        return (a > b) ? a : b;
    }

    // Method to calculate area of circle
    public static double areaCircle(double radius) {
        return Math.PI * radius * radius;
    }

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);
        int choice;

        do {

            System.out.println("\n---- STUDENT UTILITY PROGRAM ----");
            System.out.println("1. Student Information");
```

```

System.out.println("2. Even or Odd");
System.out.println("3. Largest of Three Numbers");
System.out.println("4. Grade Calculation");
System.out.println("5. Day of the Week");
System.out.println("6. Multiplication Table");
System.out.println("7. Numbers from 1 to N");
System.out.println("8. Sum of First N Natural Numbers");
System.out.println("9. Fibonacci Series");
System.out.println("10. Factorial");
System.out.println("11. Prime Number Check");
System.out.println("12. Maximum of Two Numbers");
System.out.println("13. Area of Circle");
System.out.println("14. Exit");
System.out.print("Enter your choice: ");

choice = sc.nextInt();

switch (choice) {

    case 1:
        sc.nextLine();

        System.out.print("Enter Student Name: ");
        String name = sc.nextLine();

        System.out.print("Enter Roll Number: ");
        int roll = sc.nextInt();

        System.out.print("Enter Marks in Subject 1: ");
        int m1 = sc.nextInt();

        System.out.print("Enter Marks in Subject 2: ");
        int m2 = sc.nextInt();

        System.out.print("Enter Marks in Subject 3: ");
        int m3 = sc.nextInt();

        int total = m1 + m2 + m3;
        double percentage = total / 3.0;

        System.out.println("\n----- Student Details -----");
        System.out.println("Name          : " + name);
        System.out.println("Roll No     : " + roll);
        System.out.println("Total Marks: " + total);
        System.out.println("Percentage  : " + percentage + "%");
        break;

    case 2:
        System.out.print("Enter a Number: ");
        int num = sc.nextInt();

        if (num % 2 == 0)
            System.out.println(num + " is Even.");
        else
            System.out.println(num + " is Odd.");

        break;
}

```

```

case 3:
    System.out.print("Enter Three Numbers: ");
    int a = sc.nextInt();
    int b = sc.nextInt();
    int c = sc.nextInt();

    int largest = a;

    if (b > largest)
        largest = b;

    if (c > largest)
        largest = c;

    System.out.println("Largest Number = " + largest);
    break;

case 4:
    System.out.print("Enter Percentage: ");
    double per = sc.nextDouble();

    if (per >= 90)
        System.out.println("Grade : A");
    else if (per >= 80)
        System.out.println("Grade : B");
    else if (per >= 70)
        System.out.println("Grade : C");
    else if (per >= 60)
        System.out.println("Grade : D");
    else
        System.out.println("Grade : F");

    break;

case 5:
    System.out.print("Enter Day Number (1-7): ");
    int day = sc.nextInt();

    switch (day) {
        case 1:
            System.out.println("Monday");
            break;
        case 2:
            System.out.println("Tuesday");
            break;
        case 3:
            System.out.println("Wednesday");
            break;
        case 4:
            System.out.println("Thursday");
            break;
        case 5:
            System.out.println("Friday");
            break;
        case 6:
            System.out.println("Saturday");

```

```

        break;
    case 7:
        System.out.println("Sunday");
        break;
    default:
        System.out.println("Invalid Day!");
    }

    break;

case 6:
    System.out.print("Enter Number: ");
    int table = sc.nextInt();

    for (int i = 1; i <= 10; i++) {
        System.out.println(
            table + " x " + i + " = " + (table * i)
        );
    }

    break;

case 7:
    System.out.print("Enter N: ");
    int n = sc.nextInt();

    for (int i = 1; i <= n; i++) {
        System.out.print(i + " ");
    }

    System.out.println();
    break;

case 8:
    System.out.print("Enter N: ");
    int num1 = sc.nextInt();

    int sum = 0;

    for (int i = 1; i <= num1; i++) {
        sum += i;
    }

    System.out.println("Sum = " + sum);
    break;

case 9:
    System.out.print("Enter Number of Terms: ");
    int terms = sc.nextInt();

    int first = 0;
    int second = 1;

    System.out.print("Fibonacci Series: ");

    for (int i = 1; i <= terms; i++) {
        System.out.print(first + " ");
    }

```

```

        int next = first + second;
        first = second;
        second = next;
    }

    System.out.println();
    break;

case 10:
    System.out.print("Enter Number: ");
    int factNum = sc.nextInt();

    System.out.println(
        "Factorial = " + factorial(factNum)
    );

    break;

case 11:
    System.out.print("Enter Number: ");
    int prime = sc.nextInt();

    if (isPrime(prime))
        System.out.println(prime + " is Prime.");
    else
        System.out.println(prime + " is Not Prime.");

    break;

case 12:
    System.out.print("Enter Two Numbers: ");
    int x = sc.nextInt();
    int y = sc.nextInt();

    System.out.println(
        "Maximum = " + maximum(x, y)
    );

    break;

case 13:
    System.out.print("Enter Radius: ");
    double radius = sc.nextDouble();

    System.out.println(
        "Area = " + areaCircle(radius)
    );

    break;

case 14:
    System.out.println("Thank You!");
    break;

default:
    System.out.println("Invalid Choice!");

```

```

        }

        } while (choice != 14);

        sc.close();
    }
}

```

Sample Output

```

----- STUDENT UTILITY PROGRAM -----
1. Student Information
2. Even or Odd
3. Largest of Three Numbers
4. Grade Calculation
5. Day of the Week
6. Multiplication Table
7. Numbers from 1 to N
8. Sum of First N Natural Numbers
9. Fibonacci Series
10. Factorial
11. Prime Number Check
12. Maximum of Two Numbers
13. Area of Circle
14. Exit
Enter your choice: 1

Enter Student Name: Rahul
Enter Roll Number: 101
Enter Marks in Subject 1: 85
Enter Marks in Subject 2: 90
Enter Marks in Subject 3: 80

----- Student Details -----
Name      : Rahul
Roll No   : 101
Total Marks: 255
Percentage : 85.0%

```

Example – Factorial

```

Enter your choice: 10
Enter Number: 5
Factorial = 120

```

Example – Prime Check

```

Enter your choice: 11
Enter Number: 17
17 is Prime.

```

Result

The Student Utility Program was successfully implemented using Java programming fundamentals. The program demonstrates variables, input/output, conditional statements, loops, switch, methods, and a menu-driven approach.

OBSERVATION / VIVA QUESTIONS – PROGRAM 1

1. What is the structure of a Java program?

A Java program generally contains:

1. Package declaration – optional.
2. Import statements – used to access library classes.
3. Class declaration.
4. Variables and methods.
5. `main()` method – starting point of program execution.
6. Statements and expressions.

Example:

```
package recordprograms;

import java.util.Scanner;

public class Example {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.println("Hello Java");

        sc.close();

    }

}
```

The `main()` method is the entry point from which execution begins.

2. Differentiate between primitive and non-primitive data types.

Primitive Data Types

Primitive types are the basic data types provided by Java.

Examples:

```
int age = 18;
double salary = 25000.50;
char grade = 'A';
boolean passed = true;
```

Java has eight primitive data types:

```
byte
short
int
long
float
double
char
boolean
```

Non-Primitive Data Types

Non-primitive types are reference types and can represent objects.

Examples:

```
String name = "Rahul";
int[] marks = {85, 90, 80};
Scanner sc = new Scanner(System.in);
```

Difference

Primitive	Non-Primitive
Basic data type	Reference/object type
Stores a value directly	Refers to an object
Examples: <code>int</code> , <code>double</code> , <code>char</code>	Examples: <code>String</code> , arrays, classes
Usually has fixed size	Size depends on the object

3. Compare if-else and switch statements.

if-else

if-else is useful when conditions involve comparisons or ranges.

Example from the program:

```
if (per >= 90)
    System.out.println("Grade : A");
else if (per >= 80)
    System.out.println("Grade : B");
else
    System.out.println("Grade : F");
```

switch

switch is useful when one expression needs to be compared against several fixed values.

Example:

```
switch (day) {
    case 1:
        System.out.println("Monday");
        break;

    case 2:
        System.out.println("Tuesday");
        break;

    default:
        System.out.println("Invalid Day!");
}
```

if-else

switch

Handles complex conditions Handles multiple fixed cases

Good for ranges Good for discrete values

Uses boolean expressions Uses matching case values

Suitable for grade calculation Suitable for day/menu selection

4. Explain the differences between for, while, and do-while loops.

for loop

Used when the number of iterations is generally known.

```
for (int i = 1; i <= 10; i++) {  
    System.out.println(i);  
}
```

while loop

Checks the condition before executing the loop body.

```
while (condition) {  
    // statements  
}
```

The loop may execute zero times.

do-while loop

Executes the body first and checks the condition afterward.

```
do {  
    // statements  
} while (condition);
```

Therefore, it executes at least once.

The menu in this program uses:

```
do {  
    // display menu  
} while (choice != 14);
```

This ensures that the menu is displayed at least once.

5. What are methods? Why are they useful in Java?

A method is a block of code that performs a specific task.

Methods help to:

- Reuse code.

- Reduce repetition.
- Improve readability.
- Divide a large program into smaller parts.
- Make testing and maintenance easier.

Example:

```
public static int maximum(int a, int b) {
    return (a > b) ? a : b;
}
```

The method receives two parameters and returns the larger number.

Another example:

```
public static double areaCircle(double radius) {
    return Math.PI * radius * radius;
}
```

6. Explain the difference between formal parameters and actual parameters.

Formal Parameters

Formal parameters are the variables declared in the method definition.

```
public static int maximum(int a, int b)
```

Here, `a` and `b` are formal parameters.

Actual Parameters

Actual parameters are the values passed when calling the method.

```
maximum(10, 20);
```

Here, `10` and `20` are actual parameters.

Formal Parameters	Actual Parameters
Declared in method definition	Passed during method call
Receive values	Supply values

Formal Parameters

Example: a, b

Actual Parameters

Example: 10, 20

PROGRAM 2: OBJECT-ORIENTED PROGRAMMING IN JAVA

Title

Library Management System

Duration

2 Hours

Objective

To understand and implement the core Object-Oriented Programming concepts in Java, including:

- Classes and objects
- Constructors
- Encapsulation
- Inheritance
- Method overloading
- Method overriding
- Polymorphism
- Abstraction
- Interfaces

Problem Statement

Develop a **Library Management System** to demonstrate important Object-Oriented Programming concepts.

The program should demonstrate:

Part A – Classes and Objects

Create a `Book` class containing:

- `Book ID`
- `Book Name`
- `Author`
- `Price`

Part B – Constructors and Encapsulation

- Create default and parameterized constructors.
- Declare data members as private.
- Access them using getters and setters.

Part C – Inheritance

Create:

- `Person`
- `Student`
- `Faculty`

`Student` and `Faculty` should inherit from `Person`.

Part D – Polymorphism

Demonstrate:

- Method overloading for calculating areas.
- Method overriding using `Vehicle`, `Car`, and `Bike`.

Part E – Abstraction and Interfaces

Create:

- Abstract class `Shape`
- `Circle` and `Rectangle` classes
- `Printable` interface
- `Report` implementation class

OOP Concepts Used

Concept	Implementation
Class	<code>Book</code> , <code>Person</code> , <code>Student</code> , <code>Faculty</code>
Object	<code>Book</code> , <code>Student</code> , <code>Faculty</code> objects
Constructor	<code>Book()</code> and parameterized <code>Book()</code>
Encapsulation	Private <code>Book</code> fields with getters/setters
Inheritance	<code>Student</code> extends <code>Person</code>
Method Overloading	<code>AreaCalculator.area()</code>
Method Overriding	<code>Car</code> and <code>Bike</code> override <code>display()</code>
Runtime Polymorphism	<code>Vehicle</code> reference to <code>Car/Bike</code>
Abstraction	Abstract <code>Shape</code> class
Interface	<code>Printable</code>
Implementation	<code>Report</code> implements <code>Printable</code>

Algorithm / Logic

1. Create the `Book` class.
2. Declare `Book` properties as private.
3. Create default and parameterized constructors.
4. Create getter and setter methods.
5. Create `Person` as the parent class.
6. Create `Student` and `Faculty` as child classes.
7. Override the `display()` method in both child classes.
8. Create `AreaCalculator`.
9. Overload the `area()` method for different shapes.
10. Create `Vehicle` as a parent class.
11. Override `display()` in `Car` and `Bike`.
12. Create abstract class `Shape`.
13. Create `Circle` and `Rectangle` classes that implement `draw()`.
14. Create `Printable` interface.
15. Implement the interface in `Report`.
16. Create objects in `main()`.

17. Demonstrate all OOP concepts through the objects.

Source Code

```
package recordprograms;

import java.util.Scanner;

// Library Management System demonstrating OOP concepts

// Part A & B: Classes, Objects, Constructors, Encapsulation
class Book {
    private int bookId;
    private String bookName;
    private String author;
    private double price;

    // Default Constructor
    public Book() {
        bookId = 0;
        bookName = "Unknown";
        author = "Unknown";
        price = 0.0;
    }

    // Parameterized Constructor
    public Book(int id, String name, String auth, double pr) {
        bookId = id;
        bookName = name;
        author = auth;
        price = pr;
    }

    // Getters and Setters
    public int getBookId() {
        return bookId;
    }

    public void setBookId(int id) {
        bookId = id;
    }

    public String getBookName() {
        return bookName;
    }

    public void setBookName(String name) {
        bookName = name;
    }

    public String getAuthor() {
        return author;
    }

    public void setAuthor(String auth) {
        author = auth;
    }
}
```

```

    }

    public double getPrice() {
        return price;
    }

    public void setPrice(double pr) {
        price = pr;
    }

    public void displayDetails() {
        System.out.println(bookId + " | " + bookName + " | "
            + author + " | " + price);
    }
}

// Part C: Inheritance
class Person {
    String name;
    int age;

    void display() {
        System.out.println("Name: " + name + ", Age: " + age);
    }
}

class Student extends Person {
    String course;

    @Override
    void display() {
        super.display();
        System.out.println("Course: " + course);
    }
}

class Faculty extends Person {
    String department;

    @Override
    void display() {
        super.display();
        System.out.println("Department: " + department);
    }
}

// Part D: Polymorphism
class AreaCalculator {

    // Circle
    double area(double radius) {
        return Math.PI * radius * radius;
    }

    // Rectangle
    double area(double length, double breadth) {
        return length * breadth;
    }
}

```

```

    }

    // Triangle
    double area(double base, double height, boolean triangle) {
        return 0.5 * base * height;
    }
}

// Runtime Polymorphism
class Vehicle {
    void display() {
        System.out.println("This is a Vehicle.");
    }
}

class Car extends Vehicle {
    @Override
    void display() {
        System.out.println("This is a Car.");
    }
}

class Bike extends Vehicle {
    @Override
    void display() {
        System.out.println("This is a Bike.");
    }
}

// Part E: Abstraction
abstract class Shape {
    abstract void draw();
}

class Circle extends Shape {
    @Override
    void draw() {
        System.out.println("Drawing a Circle");
    }
}

class Rectangle extends Shape {
    @Override
    void draw() {
        System.out.println("Drawing a Rectangle");
    }
}

// Interface
interface Printable {
    void print();
}

class Report implements Printable {
    @Override
    public void print() {
        System.out.println("Printing Report...");
    }
}

```

```

    }
}

// Main Class
public class LibraryManagementSystem {

    public static void main(String[] args) {

        // Part A & B
        System.out.println("=== Part A & B: Book Details ===");

        Book b1 = new Book();

        Book b2 = new Book(
            101,
            "Java Programming",
            "James Gosling",
            499.99
        );

        b1.displayDetails();
        b2.displayDetails();

        // Part C
        System.out.println("\n=== Part C: Inheritance ===");

        Student s = new Student();
        s.name = "Sufiyy";
        s.age = 18;
        s.course = "Computer Science AI ML";
        s.display();

        System.out.println();

        Faculty f = new Faculty();
        f.name = "Dr. Ram";
        f.age = 40;
        f.department = "Engineering Graphics";
        f.display();

        // Part D
        System.out.println("\n=== Part D: Polymorphism ===");

        AreaCalculator ac = new AreaCalculator();

        System.out.println("Circle Area: " + ac.area(5));
        System.out.println("Rectangle Area: " + ac.area(4, 6));
        System.out.println("Triangle Area: " + ac.area(3, 7, true));

        System.out.println();

        Vehicle v = new Vehicle();
        Vehicle car = new Car();
        Vehicle bike = new Bike();

        v.display();
        car.display();
    }
}

```

```

        bike.display();

        // Part E
        System.out.println("\n=== Part E: Abstraction & Interfaces ===");

        Shape circle = new Circle();
        Shape rectangle = new Rectangle();

        circle.draw();
        rectangle.draw();

        Report report = new Report();
        report.print();
    }
}

```

Note

The supplied program contains:

```
import java.util.Scanner;
```

but does not actually use `Scanner`. This **does not cause a compile-time error**; it is simply an unused import and may be removed.

Sample Output

```

=== Part A & B: Book Details ===
0 | Unknown | Unknown | 0.0
101 | Java Programming | James Gosling | 499.99

```

```

=== Part C: Inheritance ===
Name: Sufiyy, Age: 18
Course: Computer Science AI ML

```

```

Name: Dr. Ram, Age: 40
Department: Engineering Graphics

```

```

=== Part D: Polymorphism ===
Circle Area: 78.53981633974483
Rectangle Area: 24.0
Triangle Area: 10.5

```

```

This is a Vehicle.
This is a Car.
This is a Bike.

```

```

=== Part E: Abstraction & Interfaces ===
Drawing a Circle
Drawing a Rectangle
Printing Report...

```

Result

The Library Management System was successfully implemented using Java Object-Oriented Programming concepts including **classes, objects, constructors, encapsulation, inheritance, method overloading, method overriding, polymorphism, abstraction, and interfaces.**

OBSERVATION / VIVA QUESTIONS – PROGRAM 2

1. What is Object-Oriented Programming?

Object-Oriented Programming, or OOP, is a programming approach where programs are designed using **objects and classes.**

An object contains:

- Data/state
- Methods/behavior

Java is strongly based on OOP concepts.

Example:

```
Book b2 = new Book(  
    101,  
    "Java Programming",  
    "James Gosling",  
    499.99  
);
```

Here, Book is the class and b2 is an object.

The major OOP concepts are:

1. Encapsulation
 2. Inheritance
 3. Polymorphism
 4. Abstraction
-

2. Differentiate between a Class and an Object.

Class

A class is a blueprint or template used to create objects.

Example:

```
class Book {  
    private int bookId;  
    private String bookName;  
}
```

Object

An object is an actual instance of a class.

```
Book b1 = new Book();
```

Here:

- Book → class
- b1 → object

Class	Object
Blueprint	Instance
Defines properties and methods	Uses those properties and methods
Does not represent a specific instance	Represents a specific instance
Example: Book	Example: b1

3. Explain the purpose of constructors.

A constructor is a special member of a class used to initialize objects.

It has:

- The same name as the class.
- No return type.
- Automatic invocation when an object is created.

The program contains a default constructor:

```
public Book() {  
    bookId = 0;  
    bookName = "Unknown";  
    author = "Unknown";  
    price = 0.0;  
}
```

It also contains a parameterized constructor:

```
public Book(int id, String name, String auth, double pr) {  
    bookId = id;  
    bookName = name;  
    author = auth;  
    price = pr;  
}
```

The parameterized constructor allows values to be supplied when creating an object:

```
Book b2 = new Book(  
    101,  
    "Java Programming",  
    "James Gosling",  
    499.99  
);
```

4. What is Encapsulation? How is it implemented in Java?

Encapsulation means **bundling data and methods together while controlling direct access to the data.**

In Java, encapsulation is commonly implemented using:

- private data members
- Public getter methods
- Public setter methods

Example:

```
class Book {  
    private int bookId;  
    private double price;  
  
    public int getBookId() {  
        return bookId;  
    }  
  
    public void setBookId(int id) {  
        bookId = id;  
    }  
}
```

```
    }  
}
```

Here, `bookId` and `price` cannot be accessed directly from outside the class.

This provides better control over the object's data.

5. Differentiate between Method Overloading and Method Overriding.

Method Overloading

Method overloading occurs when multiple methods have the **same name but different parameter lists** in the same class.

Example:

```
double area(double radius) {  
    return Math.PI * radius * radius;  
}  
  
double area(double length, double breadth) {  
    return length * breadth;  
}
```

The `area()` method is overloaded.

Method Overriding

Method overriding occurs when a child class provides its own implementation of a method inherited from the parent class.

Example:

```
class Vehicle {  
    void display() {  
        System.out.println("This is a Vehicle.");  
    }  
}  
  
class Car extends Vehicle {  
    @Override  
    void display() {  
        System.out.println("This is a Car.");  
    }  
}
```

Overloading	Overriding
Same method name	Same method name
Different parameters	Same method signature
Usually within same class	Requires inheritance
Compile-time polymorphism	Runtime polymorphism

6. Explain the four pillars of OOP with examples.

1. Encapsulation

Wrapping data and methods into a class and controlling access.

Example:

```
private double price;  
  
public double getPrice() {  
    return price;  
}
```

2. Inheritance

A child class acquires properties and methods of a parent class.

Example:

```
class Student extends Person {  
}
```

Here, Student inherits from Person.

3. Polymorphism

Polymorphism means **one interface/name can have multiple forms**.

The program demonstrates method overloading:

```
area(5);  
area(4, 6);
```

It also demonstrates runtime polymorphism:

```
Vehicle car = new Car();  
car.display();
```

The Car implementation of display() is executed.

4. Abstraction

Abstraction hides implementation details and exposes only necessary functionality.

Example:

```
abstract class Shape {  
    abstract void draw();  
}
```

The subclasses provide the implementation:

```
class Circle extends Shape {  
    @Override  
    void draw() {  
        System.out.println("Drawing a Circle");  
    }  
}
```

7. Differentiate between Abstract Classes and Interfaces.

Abstract Class

An abstract class can contain abstract methods as well as concrete methods and data members.

Example:

```
abstract class Shape {  
    abstract void draw();  
}
```

Implemented by:

```
class Circle extends Shape {  
    @Override  
    void draw() {  
        System.out.println("Drawing a Circle");  
    }  
}
```

Interface

An interface defines a contract that implementing classes must follow.

Example:

```
interface Printable {  
    void print();  
}
```

Implementation:

```
class Report implements Printable {  
    @Override  
    public void print() {  
        System.out.println("Printing Report...");  
    }  
}
```

Abstract Class

Interface

Declared using `abstract class` Declared using `interface`

Extended using `extends` Implemented using `implements`

Can have concrete methods Primarily defines contracts

Can have instance fields Fields are constants

A class extends one class A class can implement multiple interfaces

PROGRAM 1 MARK DISTRIBUTION

Component	Marks
Java Program Structure	2
Conditional Statements	2
Looping Statements	2
Methods Implementation	3
Observation Answers	1
Total	10

PROGRAM 2 MARK DISTRIBUTION

Component	Marks
Classes, Objects & Constructors	2
Encapsulation & Inheritance	3
Polymorphism	2
Abstraction & Interfaces	2
Observation Answers	1
Total	10

LEARNING OUTCOMES

After completing these two tasks, the student will be able to:

- Develop basic Java programs.
- Apply decision-making and looping constructs.
- Design reusable methods.
- Understand classes and objects.
- Apply encapsulation and inheritance.
- Demonstrate compile-time and runtime polymorphism.
- Implement abstraction and interfaces.
- Develop Java applications using OOP principles.